



Universidad Nacional Experimental De Guayana

Vicerrectorado Académico.

Coordinación General De Pregrado.

Proyecto De Carrera: Ingeniería Informática

Lenguajes y Compiladores

Grupo: Control de Flujo

Lenguajes y Gramáticas Formales

Profesor:

Márquez Félix

Integrantes:

Canchis Victor: 27935276

Gonzalez Blas: 31074243

Muñoz Diosdado: 30932108

Rojas Celimar: 31981398

Ciudad Guayana, junio de 2026

Tabla de Contenido

Resumen Académico	5
Introducción	6
Fundamentos y Jerarquía (Teoría Aplicada)	7
Relación Gramática - Lenguaje.....	7
<i>Gramática Formal:</i>	<i>7</i>
Lenguaje Formal.....	8
Relación entre Gramáticas y Lenguajes Formales	8
Mecanismo mediante el cual una Gramática genera un Lenguaje	8
<i>Implementación Práctica en Java.....</i>	<i>10</i>
Jerarquía de Chomsky	11
<i>Gramáticas Tipo 0 (Irrestringidas).....</i>	<i>11</i>
<i>Gramáticas Tipo 1 (Sensibles al Contexto)</i>	<i>13</i>
<i>Gramáticas Tipo 2 (Libres de Contexto)</i>	<i>15</i>
<i>Gramáticas Tipo 3 (Regulares)</i>	<i>17</i>
Derivación y Modelado (Caso Práctico: Genoma).....	19
Definición Formal de una GLC	19
Alfabeto Terminal $\Sigma = \{a, c, g, t\}$.....	19
Conjuntos de la Gramática	20

Conjunto de Producciones P	20
Ejemplos de Derivación.....	22
<i>Caso (a): Cuadrado</i>	<i>22</i>
<i>Caso (b): Árbol con Ramas y Hojas</i>	<i>23</i>
<i>Caso (c): Cubo.....</i>	<i>24</i>
<i>Caso (d): Hexágono Regular</i>	<i>26</i>
<i>Caso (e): Espiral Rectangular.....</i>	<i>27</i>
Análisis Comparativo de las Derivaciones.....	28
Higiene y Optimización de gramática	30
Higiene de Gramáticas (Simplificación)	30
Optimización de Gramáticas	30
Patologías de las Gramáticas	31
<i>Ambigüedad</i>	<i>31</i>
<i>Caso práctico.....</i>	<i>32</i>
Rekursividad por la Izquierda	33
<i>Caso Practico.....</i>	<i>34</i>
Factorización Izquierda.....	37
<i>Caso Práctico.....</i>	<i>37</i>
De la Expresión al Autómata (Caso Práctico: Ajedrez)	42
Notación PGN (Portable Game Notation)	42

Definición de un subconjunto simplificado del PGN	43
Expresión Regular (Regex) y Autómata Finito Determinístico (AFD)	44
<i>Expresión regular</i>	44
Autómata Finito Determinístico	44
Implementación en Python y validación	46
Conclusión	49
Referencias.....	50

Resumen Académico

Este trabajo aborda los fundamentos teóricos y prácticos de los lenguajes formales y su aplicación en el diseño de compiladores. Se inicia con el estudio de la relación entre gramáticas y lenguajes, formalizando el proceso de derivación que permite generar cadenas válidas. A continuación, se analiza la Jerarquía de Chomsky, clasificando las gramáticas en cuatro tipos (0, 1, 2 y 3) e ilustrando cada uno con implementaciones prácticas en Java. El estudio se profundiza con un caso práctico de modelado de figuras geométricas mediante una Gramática Libre de Contexto (GLC) y sus derivaciones. Posteriormente, se exploran técnicas de higiene y optimización gramatical, abordando patologías como la ambigüedad, la recursividad por la izquierda y la factorización izquierda, y se demuestra su resolución mediante algoritmos y simulaciones en JavaScript. Finalmente, se aplica el concepto de autómatas finitos al diseño de un analizador léxico para un subconjunto de la notación PGN de ajedrez, implementado en Python, que valida la transición de una expresión regular a un Autómata Finito Determinístico (AFD), demostrando la aplicabilidad de la teoría en un contexto real y verificable.

Introducción

En las ciencias de la computación, los lenguajes formales y la teoría de autómatas constituyen los pilares sobre los que se construyen los compiladores, intérpretes y demás sistemas de procesamiento de lenguajes. La capacidad de definir con precisión la sintaxis de un lenguaje y de construir mecanismos eficientes para su reconocimiento es fundamental para el desarrollo de software. Este trabajo se enfoca en el estudio de estos principios, explorando desde las bases matemáticas de las gramáticas hasta la implementación de analizadores léxicos y sintácticos.

El documento se estructura en varios bloques temáticos. En primer lugar, se establecen los fundamentos y la jerarquía de las gramáticas formales según la clasificación de Chomsky, proporcionando ejemplos prácticos para cada tipo. Este análisis teórico se complementa con un caso práctico de modelado genético, donde se utiliza una Gramática Libre de Contexto para generar figuras geométricas, ilustrando el concepto de derivación y la generación de lenguajes. Luego, se abordan las técnicas de "higiene" y optimización de gramáticas, esenciales para la construcción de analizadores deterministas. Aquí se examinan patologías comunes como la ambigüedad y la recursividad por la izquierda, y se proponen soluciones para su eliminación. Para cerrar, se desarrolla un caso práctico final, donde se traduce una expresión regular en un Autómata Finito Determinístico para validar movimientos de ajedrez, demostrando la conexión directa entre la teoría de lenguajes y su aplicación práctica en un dominio específico.

Fundamentos y Jerarquía (Teoría Aplicada)

En esta sección se abordan los pilares fundamentales que rigen a las ciencias de la computación en materia de lenguajes formales. A continuación, se analiza la interacción matemática entre las gramáticas y los lenguajes que estas generan, así como la clasificación estructural propuesta en la Jerarquía de Chomsky.

Relación Gramática - Lenguaje

Gramática Formal:

Una gramática formal es un conjunto de reglas de producción que define la estructura sintáctica de un lenguaje. Estas reglas especifican cómo pueden combinarse los símbolos de un alfabeto para formar cadenas válidas. En el contexto de los lenguajes de programación, las gramáticas permiten describir la sintaxis que debe cumplir un programa para ser procesado correctamente por un compilador o intérprete.

Formalmente una gramática se representa como:

$$G = (N, \Sigma, P, S)$$

Donde:

- *N*: conjunto de símbolos no terminales.
- *Σ*: conjunto de símbolos terminales.
- *P*: conjunto de reglas de producción.
- *S*: símbolo inicial.

Lenguaje Formal

Un lenguaje formal es un conjunto de cadenas construidas a partir de un alfabeto determinado y generadas mediante una gramática formal. Cada cadena perteneciente al lenguaje cumple estrictamente las reglas sintácticas establecidas por la gramática.

Los lenguajes formales constituyen la base teórica de los lenguajes de programación, compiladores, intérpretes y sistemas de procesamiento de información.

Relación entre Gramáticas y Lenguajes Formales

La relación entre una gramática y un lenguaje formal es directa. La gramática actúa como un conjunto de instrucciones que especifica cómo generar cadenas válidas, mientras que el lenguaje formal corresponde al conjunto de todas las cadenas que pueden obtenerse aplicando dichas reglas.

En otras palabras, la gramática define la estructura y las restricciones, mientras que el lenguaje está compuesto por los resultados producidos a partir de esas reglas. Sin una gramática no existiría un criterio formal para determinar qué cadenas son válidas dentro de un lenguaje.

Mecanismo mediante el cual una Gramática genera un Lenguaje

El mecanismo mediante el cual una gramática genera un lenguaje se basa en un proceso formal denominado derivación. Este proceso consiste en aplicar sucesivamente las

reglas de producción de la gramática para transformar el símbolo inicial en cadenas válidas del lenguaje.

El proceso puede describirse mediante las siguientes etapas:

- **Inicio:** Se parte del símbolo inicial o axioma de la gramática, generalmente representado por S.
- **Aplicación de reglas de producción:** Se selecciona una regla de producción cuya parte izquierda coincida con el símbolo o secuencia actual y se reemplaza por la expresión definida en el lado derecho de la regla.
- **Derivación sucesiva:** El procedimiento se repite tantas veces como sea necesario, generando nuevas cadenas intermedias.
- **Obtención de una cadena terminal:** El proceso finaliza cuando la cadena obtenida está compuesta únicamente por símbolos terminales, es decir, símbolos que ya no pueden ser reemplazados mediante ninguna producción.

El conjunto de todas las cadenas terminales que pueden obtenerse a partir del símbolo inicial utilizando las reglas de producción constituye formalmente el lenguaje generado por la gramática, denotado como $L(G)$.

Ejemplo Práctico

Considérese la siguiente gramática:

- $S \rightarrow x$
- $S \rightarrow y$
- $S \rightarrow x+y$
- $S \rightarrow y+x$

A partir de estas producciones se generan cadenas válidas como:

- x
- y
- $x+y$
- $y+x$
- $x+x$
- $y+y$

El lenguaje generado está conformado por todas las expresiones válidas reconocidas por la gramática.

Implementación Práctica en Java

Con el propósito de demostrar la relación entre gramática y lenguaje formal, se desarrolló un programa en Java capaz de validar expresiones simples construidas con los símbolos terminales x e y y el operador $+$.

El programa verifica si la cadena ingresada pertenece al lenguaje definido por la gramática propuesta y emite un resultado de aceptación o rechazo.

```
package src_fundamentos_jerarquia;

import java.util.Scanner;

public class EjemploGeneralRelacion {

    public static boolean validar(String cadena) {

        return cadena.matches("(^ (x|y) (\\+ (x|y) ) ?$");

    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("[2.2.1] Ingrese una expresion (Ej: x+y): ");
```

```

String entrada = scanner.nextLine().trim();

System.out.println(

    validar(entrada)

        ? "OK: La cadena pertenece al lenguaje."

        : "ERROR: La cadena NO pertenece al lenguaje."

    scanner.close();

}

}

```

Jerarquía de Chomsky

La Jerarquía de Chomsky es una clasificación propuesta por Noam Chomsky en 1956 para organizar las gramáticas formales según su capacidad de generación y reconocimiento de lenguajes. Esta jerarquía establece cuatro niveles de complejidad: Tipo 0, Tipo 1, Tipo 2 y Tipo 3.

A medida que aumenta la capacidad expresiva de una gramática también aumenta la complejidad computacional requerida para procesarla.

Gramáticas Tipo 0 (Irrestrictas)

Las gramáticas irrestrictas poseen el mayor poder de expresión dentro de la jerarquía. Son equivalentes a las Máquinas de Turing y permiten describir cualquier lenguaje recursivamente enumerable.

Ejemplo Simplificado en BNF:

$::= AB$

::= Z

::= resultado

Esta representación es únicamente ilustrativa, ya que las gramáticas irrestrictas exceden las capacidades expresivas de la notación BNF clásica.

Ejemplo Práctico en Java:

Se implementó un programa que simula transformaciones arbitrarias de cadenas mediante reglas de reescritura, emulando el comportamiento general de una gramática irrestricta.

```
package src_fundamentos_jerarquia;

import java.util.Scanner;

public class ChomskyTipo0Irrestricta {

    public static boolean validarTransformacion(String cadena) {

        if (!cadena.contains("A") || !cadena.contains("B"))

            return false;

        String resultado = cadena.replace("A", "w")

            .replace("wB", "z");

        return resultado.contains("z");

    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("[Tipo 0] Ingrese cadena: ");

        String entrada = scanner.nextLine().trim();

        System.out.println(
```

```

        validarTransformacion(entrada)

        ? "OK: Transformación aceptada."

        : "ERROR: Transformación rechazada.");

    scanner.close();

}

}

```

Gramáticas Tipo 1 (Sensibles al Contexto)

Las gramáticas sensibles al contexto permiten expresar dependencias entre diferentes partes de una cadena, donde la validez de un símbolo puede depender de los símbolos que lo rodean.

Representación Formal

$$L = \{ a^n b^n c^n \mid n \geq 1 \}$$

Ejemplos válidos:

- abc
- aabbcc
- aaabbbccc

Debido a que las restricciones de contexto no pueden representarse completamente mediante BNF tradicional, se presenta la definición formal del lenguaje junto con una implementación práctica.

Ejemplo Práctico en Java

Se desarrolló un programa que verifica que la cantidad de símbolos a, b y c sea exactamente la misma y aparezcan en el orden correcto.

```
package src_fundamentos_jerarquia;

import java.util.Scanner;

public class ChomskyTipolSensibleContexto {

    public static boolean validar(String cadena) {

        int a = 0, b = 0, c = 0, i = 0;

        while (i < cadena.length() && cadena.charAt(i) == 'a') {

            a++;

            i++;

        }

        while (i < cadena.length() && cadena.charAt(i) == 'b') {

            b++;

            i++;

        }

        while (i < cadena.length() && cadena.charAt(i) == 'c') {

            c++;

            i++;

        }

        return i == cadena.length()

            && a > 0

            && a == b

            && b == c;

    }

}
```

```

public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);

    System.out.print("[Tipo 1] Ingrese cadena: ");

    String entrada = scanner.nextLine().trim();

    System.out.println(

        validar(entrada)

            ? "[OK] -> Pertenece al lenguaje."

            : "[ERROR] -> No pertenece al lenguaje.");

    scanner.close();

}
}

```

Gramáticas Tipo 2 (Libres de Contexto)

Las gramáticas libres de contexto permiten modelar estructuras jerárquicas y anidadas como expresiones matemáticas, bloques de código y paréntesis balanceados.

Ejemplo en Notación BNF

$::= a$

$::= b$

$::= ()$

Ejemplo Práctico en Java

Se implementó un analizador sintáctico por descenso recursivo capaz de verificar expresiones con paréntesis balanceados.

Cadenas válidas:

- $x=a$
- $x=(a)$
- $x=((b))$

```
package src_fundamentos_jerarquia;

import java.util.Scanner;

public class ChomskyTipo2LibreContexto {

    public static boolean validar(String cadena) {

        if (!cadena.startsWith("x="))

            return false;

        return evaluar(cadena.substring(2));

    }

    private static boolean evaluar(String expr) {

        if (expr.equals("a") || expr.equals("b"))

            return true;

        if (expr.startsWith("(") && expr.endsWith(")")) {

            return evaluar(expr.substring(1, expr.length() - 1));

        }

        return false;

    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("[Tipo 2] Ingrese expresión: ");

        String entrada = scanner.nextLine().trim();

        System.out.println(
```



```

        validar(entrada)

            ? "[OK] -> Expresion valida."

            : "[ERROR] -> Expresion invalida."

    );

    scanner.close();

}

}

```

Gramáticas Tipo 3 (Regulares)

Las gramáticas regulares constituyen el nivel más simple de la jerarquía. Son ampliamente utilizadas en el análisis léxico de compiladores para reconocer identificadores, palabras reservadas y constantes.

Ejemplo en Notación BNF

`::= id | id1`

Ejemplo Práctico en Java

Se desarrolló un programa capaz de reconocer identificadores simples utilizados dentro de un sistema de software.

Cadenas válidas:

- id
- id1

```

package src_fundamentos_jerarquia;

import java.util.Scanner;

```

```

public class ChomskyTipo3Regular {

    public static boolean validar(String cadena) {

        return cadena.matches("^id1?$");

    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("[Tipo 3] Ingrese un identificador: ");

        String entrada = scanner.nextLine().trim();

        System.out.println(

            validar(entrada)

                ? "OK: Cadena válida."

                : "ERROR: Cadena inválida.");

        scanner.close();

    }

}

```

Finalmente, el estudio de la relación entre gramáticas y lenguajes formales permitió comprender cómo las reglas de producción sirven como mecanismo para generar y validar cadenas dentro de un lenguaje. A partir de ello, el análisis de la Jerarquía de Chomsky mostró los distintos niveles de capacidad expresiva de las gramáticas, desde aquellas destinadas al reconocimiento de patrones simples hasta las que permiten describir estructuras más complejas. Los ejemplos desarrollados y su implementación en Java facilitaron la comprensión práctica de estos conceptos, evidenciando su aplicación en el procesamiento y análisis de lenguajes utilizados en compiladores y otras herramientas informáticas.

Derivación y Modelado (Caso Práctico: Genoma)

Definición Formal de una GLC

Una Gramática Libre de Contexto G se define como una cuádrupla:

$$G = (V, \Sigma, P, S)$$

Donde cada componente tiene el siguiente significado:

- V es el conjunto finito de símbolos no terminales (variables).
- Σ es el alfabeto de símbolos terminales ($\Sigma \cap V = \emptyset$).
- P es el conjunto finito de producciones de la forma $A \rightarrow \alpha$, con $A \in V$ y $\alpha \in (V \cup \Sigma)^*$.
- $S \in V$ es el símbolo inicial o axioma de la gramática.

Alfabeto Terminal $\Sigma = \{a, c, g, t\}$

Para la gramática de dibujo propuesta, cada símbolo terminal representa una acción primitiva sobre el cursor o herramienta de dibujo:

<i>Símbolo</i>	<i>Acción de Dibujo</i>	<i>Ejemplo de Uso</i>
<i>a</i>	Avanzar una unidad trazando línea	Lado de polígono, segmento, rama
<i>c</i>	Girar a la derecha 60° (clockwise)	Vértice de hexágono
<i>g</i>	Girar a la derecha 90° (clockwise)	Vértice de cuadrado, esquina de cubo

t	Terminar el trazo actual (cierre de figura o de rama)	Último símbolo de toda cadena derivada
----------	---	--

Conjuntos de la Gramática

- Símbolos no terminales: $V = \{S, F, Q, H, B, R, R1, D, D1, K\}$
- Alfabeto terminal: $\Sigma = \{a, c, g, t\}$
- Símbolo inicial: S

Conjunto de Producciones P

A continuación se presenta el conjunto completo de producciones de la gramática. A diferencia de un listado de cadenas fijas, estas producciones son recursivas: cada figura se construye aplicando una regla sobre sí misma un número determinado de veces hasta alcanzar la condición de paro, lo que permite construir un árbol de derivación genuino para cada caso.

NT	→	Producción	Descripción
S	→	$F \mid H \mid C \mid B \mid K$	<i>Axioma: elige la figura a derivar</i>
F	→	$a g F \mid a t$	<i>Cuadrado (recursiva, 90° por vértice)</i>
H	→	$a c H \mid a t$	<i>Hexágono (recursiva, 60° por vértice)</i>
Q	→	$a g Q \mid a g$	<i>Cuadrado abierto (auxiliar, sin cierre, usado en C)</i>
C	→	$Q a g Q a g Q a t$	<i>Cubo: tres caras Q reposicionadas + cierre</i>

B	→ $a R D \mid a t$	<i>Árbol: bifurca en dos no terminales (rama A/B) o termina en hoja</i>
R	→ $g a R1$	<i>Rama A: fija dirección con un único giro de 90° y entra a R1</i>
R1	→ $a R1 \mid a t$	<i>Continúa la rama A en línea recta (dirección ya fijada)</i>
D	→ $g g a D1$	<i>Rama B: fija dirección con dos giros de 90° (180°) y entra a D1</i>
D1	→ $a D1 \mid a t$	<i>Continúa la rama B en línea recta (dirección ya fijada)</i>
K	→ $a a g K \mid a t$	<i>Espiral rectangular (recursiva, lado +2 por vuelta)</i>

Nótese que F y H comparten exactamente la misma estructura recursiva ($a X \mid a t$), y se diferencian únicamente por el terminal de giro que emplean (g para 90°, c para 60°). Esto es consistente con el principio de semántica fija: el ángulo de cada figura no se logra reinterpretando un terminal, sino eligiendo qué terminal de giro participa en la producción.

El mismo principio aplica a las ramas del árbol: dado que Σ solo define giros hacia un sentido (60° y 90°), no existe un terminal dedicado a 'girar a la izquierda'. La diferencia entre ambas ramas se logra fijando una dirección distinta al inicio de cada una mediante el mismo terminal g (90°, ya establecido para figuras angulares): R gira una sola vez (apunta a 90° respecto al tronco) y D gira dos veces (apunta a 180° respecto al tronco). Una vez fijada esa dirección, los no terminales auxiliares R1 y D1 solo avanzan en línea recta sin volver a girar;

si el giro se repitiera en cada nivel de recursión, dos aplicaciones de 180° se cancelarían entre sí y la rama D terminaría apuntando de nuevo hacia el tronco, lo cual fue corregido separando 'fijar dirección' (R, D) de 'avanzar en esa dirección' (R1, D1) en producciones distintas.

Ejemplos de Derivación

A continuación se presentan los cinco casos de derivación solicitados. Cada derivación parte del símbolo inicial S y aplica producciones paso a paso hasta obtener una cadena de símbolos terminales perteneciente a Σ^* .

Caso (a): Cuadrado

Un cuadrado se construye aplicando la producción recursiva F exactamente tres veces (uno por cada lado pendiente) y deteniendo la recursión en el cuarto lado, que cierra la figura. Cada paso de la derivación reescribe únicamente el no terminal F presente en la forma sentencial anterior; en ningún momento se inserta un símbolo que no provenga de esa sustitución.

Cadena generada: $a g a g a g a t$

Paso	Forma Sentencial	Regla Aplicada
0	S	Axioma (símbolo inicial)
1	F	$S \rightarrow F$
2	$a g F$	$F \rightarrow a g F$ (lado 1)

3	$a g a g F$	$F \rightarrow a g F$ (lado 2)
4	$a g a g a g F$	$F \rightarrow a g F$ (lado 3)
5	$a g a g a g a t$	$F \rightarrow a t$ (lado 4, condición de paro)

Interpretación gráfica: el cursor avanza y gira 90° (a g) tres veces para construir tres lados consecutivos, y en el cuarto lado se aplica la regla de paro (a t), que añade el último avance y cierra el trazo sin girar de nuevo, ya que el siguiente vértice coincide con el punto de partida.

Caso (b): Árbol con Ramas y Hojas

El árbol se modela con cinco no terminales que trabajan en conjunto: B inicia el tronco y bifurca en R y D, cada uno de los cuales fija una dirección distinta mediante un único giro (R gira 90° , D gira 180° , usando siempre el mismo terminal g) y delega en un no terminal auxiliar (R1 y D1, respectivamente) que avanza en línea recta en esa dirección hasta cerrar en una hoja. Separar 'fijar dirección' de 'avanzar' evita que giros repetidos en cada nivel de recursión se cancelen entre sí: si D girara 180° en cada aplicación recursiva, dos aplicaciones sucesivas devolverían el ángulo a su valor original y la rama terminaría apuntando de nuevo hacia el tronco. Al fijar la dirección una sola vez y mantenerla constante en R1/D1, ambas ramas conservan una orientación estable y distinta entre sí.

Cadena generada: a g a a a t g g a a a t

Paso	Forma Sentencial	Regla Aplicada
0	S	<i>Axioma</i>
1	B	$S \rightarrow B$
2	$a R D$	$B \rightarrow a R D$ (<i>tronco; bifurca en R y D</i>)
3	$a g a R1 D$	$R \rightarrow g a R1$ (<i>fija dirección de R: gira 90° y avanza</i>)
4	$a g a a R1 D$	$R1 \rightarrow a R1$ (<i>la rama R continúa recto</i>)
5	$a g a a a t D$	$R1 \rightarrow a t$ (<i>la rama R cierra en hoja</i>)
6	$a g a a a t g g a D1$	$D \rightarrow g g a D1$ (<i>fija dirección de D: gira 180° y avanza</i>)
7	$a g a a a t g g a a D1$	$D1 \rightarrow a D1$ (<i>la rama D continúa recto</i>)
8	$a g a a a t g g a a a t$	$D1 \rightarrow a t$ (<i>la rama D cierra en hoja</i>)

Interpretación gráfica: el primer símbolo a traza el tronco; a partir de ahí, R gira 90° y avanza en línea recta hacia un lado, mientras que D gira 180° y avanza en línea recta en la dirección opuesta al tronco. El resultado es una bifurcación visualmente real, verificada geométricamente: la rama R se extiende en una dirección y la rama D en otra completamente distinta, ambas partiendo del mismo punto de bifurcación.

Caso (c): Cubo

El cubo se construye combinando tres caras cuadradas mediante el no terminal auxiliar Q , una variante de F que no incluye el símbolo de cierre t , ya que sus tres apariciones deben quedar conectadas entre sí mediante reposicionamientos ($a g$) antes de que la figura completa se cierre una sola vez, al final de la producción C . A continuación se desglosa cada sustitución de Q en su propia fila, sin agrupar pasos, para que la derivación sea completamente transparente: cada cara emplea exactamente tres aplicaciones recursivas de $Q \rightarrow a g Q$ (lados 1 a 3) seguidas de una aplicación de la regla de paro $Q \rightarrow a g$ (lado 4), totalizando cuatro lados por cara.

Cadena generada: $a g a g a g a g a g a g a g a g a g a g a g a g a t$

Paso	Forma Sentencial	Regla Aplicada
0	S	<i>Axioma</i>
1	C	$S \rightarrow C$
2	$Q a g Q a g Q a t$	$C \rightarrow Q a g Q a g Q a t$
3	$a g Q a g Q a g Q a t$	$Q \text{ (cara 1)} \rightarrow a g Q \text{ (lado 1)}$
4	$a g a g Q a g Q a g Q a t$	$Q \text{ (cara 1)} \rightarrow a g Q \text{ (lado 2)}$
5	$a g a g a g Q a g Q a g Q a t$	$Q \text{ (cara 1)} \rightarrow a g Q \text{ (lado 3)}$
6	$a g a g a g a g a g Q a g Q a t$	$Q \text{ (cara 1)} \rightarrow a g \text{ (lado 4, cierre parcial de la cara 1)}$
7	$agagagag ag a g Q a g Q a t$	$Q \text{ (cara 2)} \rightarrow a g Q \text{ (lado 1)}$

8	<i>agagagag ag a g a g Q a g Q a t</i>	<i>Q (cara 2) → a g Q (lado 2)</i>
9	<i>agagagag ag a g a g a g Q a g Q a t</i>	<i>Q (cara 2) → a g Q (lado 3)</i>
10	<i>agagagag ag agagagag a g Q a t</i>	<i>Q (cara 2) → a g (lado 4, cierre parcial de la cara 2)</i>
11	<i>agagagag ag agagagag ag a g Q a t</i>	<i>Q (cara 3) → a g Q (lado 1)</i>
12	<i>agagagag ag agagagag ag a g a g Q a t</i>	<i>Q (cara 3) → a g Q (lado 2)</i>
13	<i>agagagag ag agagagag ag a g a g a g Q a t</i>	<i>Q (cara 3) → a g Q (lado 3)</i>
14	<i>agagagag ag agagagag ag agagagag a t</i>	<i>Q (cara 3) → a g (lado 4, cierre parcial de la cara 3) + a t final</i>

Interpretación gráfica: cada bloque agagagag corresponde a una cara cuadrada abierta (cuatro lados sin cierre individual); el segmento a g entre caras reposiciona el cursor para iniciar la siguiente cara en la perspectiva isométrica del cubo, y el cierre a t final consolida todo el trazo en un único contorno conectado. La longitud total de la cadena terminal es de 30 símbolos: 15 avances (a) y 14 giros (g), más el cierre a t ya contado dentro de esos 15 avances.

Caso (d): Hexágono Regular

El hexágono regular utiliza la misma estructura recursiva que el cuadrado, pero con el terminal de giro c (60° fijo, según la definición del alfabeto en la sección 2.2) en lugar de g.

No se redefine el valor de ningún terminal: simplemente se emplea el símbolo de giro adecuado para el ángulo exterior de un hexágono ($360^\circ/6 = 60^\circ$).

Cadena generada: *a c a c a c a c a t*

Paso	Forma Sentencial	Regla Aplicada
0	<i>S</i>	<i>Axioma</i>
1	<i>H</i>	<i>S → H</i>
2	<i>a c H</i>	<i>H → a c H (lado 1)</i>
3	<i>a c a c H</i>	<i>H → a c H (lado 2)</i>
4	<i>a c a c a c H</i>	<i>H → a c H (lado 3)</i>
5	<i>a c a c a c a c H</i>	<i>H → a c H (lado 4)</i>
6	<i>a c a c a c a c a c H</i>	<i>H → a c H (lado 5)</i>
7	<i>a c a c a c a c a c a t</i>	<i>H → a t (lado 6, condición de paro)</i>

Interpretación gráfica: cinco aplicaciones recursivas de *a c* construyen cinco lados con giro de 60° entre cada uno; la sexta y última aplicación de la regla de paro (*a t*) añade el lado final y cierra el trazo, completando los 360° de la figura sin necesidad de un giro adicional en el último vértice.

Caso (e): Espiral Rectangular

La espiral rectangular reutiliza el patrón recursivo de F, pero avanzando dos unidades (a a) en lugar de una antes de cada giro de 90°, lo que produce lados sucesivamente más separados y genera el efecto de expansión hacia afuera característico de una espiral.

Cadena generada: a a g a a g a a g a t

Paso	Forma Sentencial	Regla Aplicada
0	S	<i>Axioma</i>
1	K	$S \rightarrow K$
2	$a a g K$	$K \rightarrow a a g K$ (tramo 1)
3	$a a g a a g K$	$K \rightarrow a a g K$ (tramo 2)
4	$a a g a a g a a g K$	$K \rightarrow a a g K$ (tramo 3)
5	$a a g a a g a a g a t$	$K \rightarrow a t$ (tramo final, condición de paro)

Interpretación gráfica: cada bloque a a g avanza dos unidades y gira 90°, generando cuatro tramos de la espiral; la profundidad de la recursión (número de veces que se aplica $K \rightarrow a a g K$ antes del paro) determina cuántas vueltas tiene la espiral, lo que demuestra la ventaja de una producción recursiva frente a una cadena fija: el mismo no terminal puede generar espirales de cualquier tamaño simplemente variando el número de derivaciones aplicadas.

Análisis Comparativo de las Derivaciones

La siguiente tabla resume las características de cada derivación presentada:

Figura	Producción	Longitud cadena	Pasos deriv.	Característica
Cuadrado	$S \rightarrow F$ (recursiva)	10 símbolos	5 pasos	Recursión simple, 1 no terminal
Árbol	$S \rightarrow B \rightarrow R, D \rightarrow RI, DI$	12 símbolos	8 pasos	Bifurcación con dirección fijada una sola vez por rama
Cubo	$S \rightarrow C \rightarrow 3Q$	30 símbolos	14 pasos	Composición de 3 caras, desglose completo sin pasos agrupados
Hexágono	$S \rightarrow H$ (recursiva)	12 símbolos	7 pasos	Misma estructura que F, terminal de giro distinto
Espiral	$S \rightarrow K$ (recursiva)	11 símbolos	5 pasos	Recursión con avance múltiple por tramo

Higiene y Optimización de gramática

La higiene y optimización de gramáticas son procesos críticos en el diseño de compiladores. Consisten en depurar y transformar las reglas sintácticas (gramáticas libres de contexto) para eliminar redundancias y ambigüedades. Esto garantiza que los analizadores sintácticos (parsers) procesen el código de forma rápida, determinista y sin errores.

Higiene de Gramáticas (Simplificación)

Para que algoritmos como LL(k) o LR(k) funcionen limpiamente, la gramática debe pasar por un proceso de "poda":

1. Eliminación de Símbolos Inútiles: Se eliminan variables o terminales que nunca se alcanzan desde el símbolo inicial o que no derivan en ninguna cadena de terminales.
2. Eliminación de Producciones: Se erradican las reglas que producen cadenas vacías, ya que complican el árbol de derivación.
3. Eliminación de Producciones Unitarias: Se eliminan reglas de la forma $A \rightarrow B$ (donde un no terminal se reemplaza por otro), reduciendo la profundidad innecesaria en el análisis.

Optimización de Gramáticas

Una gramática higiénica aún puede ser ineficiente para un compilador. Se aplican transformaciones estructurales para evitar el backtracking (retroceso):

- **Factorización por la Izquierda:** Si dos reglas comparten el mismo prefijo, se extrae el prefijo común para retrasar la decisión de qué regla aplicar hasta tener más contexto.
- **Eliminación de Recursividad por la Izquierda:** Los analizadores predictivos (como LL (1)) entran en bucles infinitos con reglas recursivas por la izquierda ($A \rightarrow A\alpha$). Se reescriben de forma iterativa usando recursividad por la derecha o notación EBNF.

Patologías de las Gramáticas

Las "patologías" o anomalías en las gramáticas formales dentro de los lenguajes y compiladores son defectos estructurales que impiden el análisis sintáctico correcto (parsing), generando bucles infinitos o ambigüedades. Las tres principales patologías son la ambigüedad, la recursividad por la izquierda y el no determinismo (factorización izquierda).

Ambigüedad

Ocurre cuando una cadena de entrada puede generar más de un árbol de derivación sintáctico válido.

- **Impacto en el compilador:** El compilador no puede determinar qué semántica aplicar, lo que genera resultados impredecibles en tiempo de ejecución. El caso más clásico es el problema del else colgante (dangling-else) donde un else no sabe a qué if asociarse.
- **Solución:** Reescribir la gramática para establecer precedencia y asociatividad, o utilizar directivas de resolución de conflictos en herramientas como Manual Bison.

Caso práctico

```
// — 1. GRAMÁTICA AMBIGUA —————

// G:  $E \rightarrow E + E \mid E * E \mid (E) \mid id$ 

// Cadena problema: 2 + 3 * 4 (id = 2, 3, 4)

console.log('=====');

console.log(' 1. DEMOSTRACIÓN DE GRAMÁTICA AMBIGUA');

console.log('=====\\n');

console.log('Gramática original:');

console.log('  $E \rightarrow E + E \mid E * E \mid (E) \mid id$ ');

console.log('Cadena de entrada: 2 + 3 * 4\\n');

function makeTree(shape) {

    if (typeof shape === 'number') return { val: shape };

    return { op: shape[1], left: makeTree(shape[0]), right:
makeTree(shape[2]) };

}

// Árbol 1 – Asociación: 2 + (3 * 4) → multiplicación dentro del subárbol
derecho

const treeMulFirst = makeTree([2, '+', [3, '*', 4]]);

// Árbol 2 – Asociación: (2 + 3) * 4 → suma dentro del subárbol izquierdo

const treeAddFirst = makeTree([[2, '+', 3], '*', 4]);

function evalTree(node) {

    if (node.val !== undefined) return node.val;

    if (node.op === '+') return evalTree(node.left) + evalTree(node.right);

    if (node.op === '*') return evalTree(node.left) * evalTree(node.right);
```



```

}

console.log('► Árbol de derivación 1 – Asociación: 2 + (3 * 4)');
console.log('  (multiplicación dentro del subárbol derecho)');
console.log('  Parse tree:', inspect(treeMulFirst));
console.log('  Resultado:', evalTree(treeMulFirst), '\n');
console.log('► Árbol de derivación 2 – Asociación: (2 + 3) * 4');
console.log('  (suma dentro del subárbol izquierdo)');
console.log('  Parse tree:', inspect(treeAddFirst));
console.log('  Resultado:', evalTree(treeAddFirst), '\n');

console.log('=> ;La misma cadena produce dos árboles y dos resultados distintos!');

console.log('=> La gramática es AMBIGUA.\n');

```

La ejecución del script concluye exitosamente la hipótesis: una gramática que permite múltiples árboles de derivación válidos para una misma cadena es inherentemente defectuosa para el diseño de lenguajes de programación. El motor de ejecución (representado por `evalTree`) realiza su trabajo de forma mecánica y determinista sobre la estructura de datos que recibe; por lo tanto, la responsabilidad de garantizar un único flujo lógico (precedencia y asociatividad) recae estrictamente en la fase previa, haciendo obligatoria la optimización y estratificación de la gramática para eliminar la ambigüedad antes de la construcción del parser.

Rekursividad por la Izquierda

Sucede cuando una regla de producción tiene a su variable no terminal como el primer símbolo a la izquierda en su propio cuerpo (ej. $A \rightarrow A \alpha$).

- **Impacto en el compilador:** Los analizadores sintácticos descendentes (Top-Down, como LL(k)) entran en un bucle recursivo infinito al intentar expandir el símbolo sin consumir ningún token.
- **Solución:** Eliminarla transformando la regla recursiva en una derivación por la derecha utilizando una nueva variable auxiliar.

Caso Practico

Este código tiene como finalidad demostrar la incompatibilidad entre las gramáticas recursivas por la izquierda y los analizadores sintácticos descendentes (Top-Down o predictivos). Mediante una simulación en dos fases, el script ilustra primero el fallo catastrófico (bucle infinito) que ocurre al intentar procesar directamente la regla $E \rightarrow E + T$ y posteriormente implementa una solución algorítmica funcional basada en la gramática optimizada, demostrando un análisis exitoso de la secuencia de tokens léxicos.

```
// — 2. RECURSIVIDAD POR LA IZQUIERDA (DEMO CONTROLADA) —————
console.log('=====');
console.log('  2. RECURSIVIDAD POR LA IZQUIERDA');
console.log('=====\\n');
console.log('Gramática original (recursiva izquierda):');
console.log('  E → E + T | T');
console.log('  T → T * F | F');
console.log('  F → (E) | id\\n');
console.log('► Simulación del bucle infinito (con protección):');
```

```

function simulateLeftRecursion(maxDepth) {

  let depth = 0;

  function parseE() {

    depth++;

    if (depth > maxDepth) throw new Error(`BUCLE INFINITO! Profundidad
máxima (${maxDepth}) alcanzada al expandir  $E \rightarrow E + T$ `);

    //  $E \rightarrow E + T$  (recursiva izquierda: se llama a sí misma antes de
consumir input)

    let savedDepth = depth;

    try {

      parseE(); // ← esto nunca avanza, llama de nuevo

    } catch (e) {

      depth = savedDepth;

      throw e;

    }

    return true;

  }

  return parseE;
}

try {

  const badParser = simulateLeftRecursion(5);

  badParser();

} catch (e) {

  console.log(' Error:', e.message, '\n');

}

console.log('► Aplicando algoritmo de eliminación:');

```

```

console.log(' E → T E\''');
console.log(' E\' → + T E\' | ε');
console.log(' T → F T\''');
console.log(' T\' → * F T\' | ε');
console.log(' F → (E) | id\n');

function parseWithoutLeftRecursion(input) {
  let pos = 0;

  const t = () => input[pos];

  function parseE() {
    parseT();

    while (t() === '+') { pos++; parseT(); }
  }

  function parseT() {
    parseF();

    while (t() === '*') { pos++; parseF(); }
  }

  function parseF() {
    if (t() === '(') { pos++; parseE(); pos++; }

    else if (/\\d+/.test(t())) { pos++; }

    else throw new Error(`Token inesperado: ${t()}`);
  }

  parseE();

  if (pos !== input.length) throw new Error(`Tokens sin consumir:
  ${input.slice(pos).join(' ')}`);

  return 'Análisis completado sin errores';
}

```

```

}

const testTokens = [2, '+', 3, '*', 4];

console.log(` Parseando [${testTokens.join(', ')}]...`);

console.log(' Resultado:', parseWithoutLeftRecursion(testTokens), '\n');

```

El bloque de control estructurado (Try-Catch) valida exitosamente la hipótesis teórica: la gramática original colapsa instantáneamente, mientras que el analizador alimentado con la gramática higienizada y libre de recursividad izquierda logra procesar el arreglo [2, '+', 3, '*', 4] hasta el final del archivo/arreglo. Esto ratifica que la optimización de la gramática no es un paso opcional,

sino un pre-requisito estricto para la viabilidad de cualquier compilador o intérprete basado en un enfoque Top-Down.

Factorización Izquierda

Se presenta cuando dos o más producciones de un mismo no terminal empiezan con la misma cadena de símbolos.

- **Impacto en el compilador:** El analizador sintáctico no sabe qué camino tomar al procesar el prefijo α , impidiendo que la gramática sea LL (1) predictiva.
- **Solución:** Factorizar el prefijo común creando un nuevo no terminal para la parte divergente

Caso Práctico

Este tercer código tiene como objetivo ilustrar y resolver uno de los problemas clásicos en el diseño de compiladores: la incapacidad de un analizador sintáctico predictivo

LL(1) para tomar decisiones deterministas cuando múltiples producciones de un mismo no-terminal comparten un prefijo común. El script demuestra este fallo utilizando la conocida ambigüedad del "Dangling Else" (el else colgante) y posteriormente aplica la técnica de Factorización por la Izquierda para adecuar la gramática, validando el resultado mediante dos implementaciones de analizadores de descenso recursivo.

```
// — 3. FACTORIZACIÓN POR LA IZQUIERDA —————

console.log('=====');

console.log(' 3. FACTORIZACIÓN POR LA IZQUIERDA');

console.log('=====\\n');

console.log('Gramática original:');

console.log(' S → if E then S else S | if E then S | other');

console.log(' Problema: prefijo común "if E then S" imposibilita decisión LL(1)\\n');

console.log('► Cálculo de conjuntos FIRST (colisión):');

console.log(' FIRST(if E then S else S) = { if }');

console.log(' FIRST(if E then S)          = { if }');

console.log(' FIRST(other)                  = { other }');

console.log(' → FIRST(if E then S else S) ∩ FIRST(if E then S) = { if } ≠ ∅');

console.log(' → El parser LL(1) NO PUEDE decidir qué producción usar\\n');

console.log('► Aplicando factorización:');

console.log(' S → if E then S S\\' | other');

console.log(' S\\' → else S | ε\\n');

console.log('► Nuevos conjuntos FIRST (disjuntos):');

console.log(' FIRST(if E then S S\\') = { if }');
```

```

console.log('  FIRST(other)                = { other }');
console.log('  FIRST(S\')                  = { else }');
console.log(' → Todos disjuntos → parser LL(1) puede decidir\n');

function parseFactored(tokens) {
  let pos = 0;

  const t = () => tokens[pos];

  function parseS() {
    if (t() === 'if') {
      pos++; // consume 'if'

      parseE();

      if (t() !== 'then') throw new Error('Se esperaba then');

      pos++;

      parseS();

      parseSPrime();
    } else if (t() === 'other') {
      pos++;
    } else {
      throw new Error(`Token inesperado: ${t()}`);
    }
  }

  function parseSPrime() {
    if (t() === 'else') {
      pos++;

      parseS();
    }
  }
}

```

```

    // ε: no hacer nada
}

function parseE() {
    if (t() === 'cond') { pos++; return; }

    throw new Error(`Token inesperado en E: ${t()}`);
}

parseS();

if (pos !== tokens.length) throw new Error('Tokens sin consumir');

return 'Análisis exitoso';
}

const ifTokens = ['if', 'cond', 'then', 'if', 'cond', 'then', 'other',
'else', 'other'];

console.log(` Parseando [${ifTokens.join(', ')}]...`);

console.log(' Resultado:', parseFactored(ifTokens), '\n');

console.log('► Segundo ejemplo (expresiones aritméticas):');

console.log(' A → id + A | id * A | id');

console.log(' Prefijo común: id\n');

console.log(' Gramática factorizada:');

console.log(' A → id A\'' );

console.log(' A\'' → +A | *A | ε\n');

function parseFactoredA(tokens) {
    let pos = 0;

    const t = () => tokens[pos];

    function parseA() {
        if (t() === 'id') { pos++; parseAPrime(); }
    }

```



```

    else throw new Error('Se esperaba id');
  }

  function parseAPrime() {

    if (t() === '+') { pos++; parseA(); }

    else if (t() === '*') { pos++; parseA(); }

    // ε: fin

  }

  parseA();

  if (pos !== tokens.length) throw new Error('Tokens sin consumir');

  return 'Análisis exitoso';

}

const aTokens = ['id', '+', 'id', '*', 'id'];

console.log(`  Parseando [${aTokens.join(', ')}]...`);

console.log('  Resultado:', parseFactoredA(aTokens), '\n');
```

Esto demuestra concluyentemente que la factorización por la izquierda, combinada con la eliminación de recursividad izquierda constituye el flujo de trabajo de "higiene gramatical" indispensable para transformar una especificación humana teórica en una arquitectura de software robusta, determinista y mecánicamente analizable por un autómata LL(1).

De la Expresión al Autómata (Caso Práctico: Ajedrez)

La validación léxica de lenguajes de dominio específico constituye un escenario ideal para demostrar la utilidad de las expresiones regulares y su conversión directa en autómatas finitos. El procesamiento de partidas de ajedrez en formato texto exige un reconocimiento robusto de cada movimiento, tarea que puede resolverse con un autómata finito determinístico (AFD) derivado de una expresión regular. En este apartado se aborda el diseño, la implementación y la validación de un analizador léxico para un subconjunto de la notación estándar PGN, mostrando el paso de la expresión regular al AFD que reconoce cadenas de movimientos válidos.

Notación PGN (Portable Game Notation)

La *Portable Game Notation* (PGN) es un formato de texto plano estandarizado a nivel mundial para el registro y el intercambio de partidas de ajedrez. Cada jugada se representa mediante una cadena de caracteres que combina la letra de la pieza, la casilla de destino y, opcionalmente, indicaciones de captura, jaque, enroque o promoción. Para que un programa pueda interpretar de forma fiable una partida, es imprescindible que su analizador léxico (scanner) sea capaz de rechazar cualquier secuencia que no se ajuste a la gramática, evitando errores en etapas posteriores del compilador o del motor de ajedrez. Un autómata finito permite realizar este reconocimiento de manera eficiente, secuencial y sin retrocesos.

La especificación completa del PGN es rica y admite construcciones como capturas ($\times$), jaque (+), jaque mate (#), enroques ($\circ-\circ$, $\circ-\circ-\circ$), promociones ($=\circ$) y

desambiguaciones de pieza ($Nbd7$). Para centrar el estudio en los mecanismos fundamentales de construcción de autómatas sin añadir una complejidad innecesaria, se define y se implementa un subconjunto simplificado que conserva la esencia del lenguaje: la expresión de movimientos básicos de una pieza o de un peón a una casilla del tablero.

Definición de un subconjunto simplificado del PGN

El subconjunto léxico que se analiza está restringido a movimientos simples, sin capturas, jaques, promociones ni elementos especiales. Las reglas sintácticas se definen formalmente como:

- **Alfabeto del subconjunto (Σ):** Unión de los símbolos correspondientes a las piezas, las columnas y las filas del tablero.
- **Piezas (Σ_P):** {K, Q, R, B, N} (Rey, Dama, Torre, Alfil, Caballo). Los peones no se denotan con letra, por lo que su ausencia indica un movimiento de peón.
- **Columnas (Σ_F):** {a, b, c, d, e, f, g, h}.
- **Filas (Σ_R):** {1, 2, 3, 4, 5, 6, 7, 8}.
- **Regla sintáctica:** Un movimiento válido consta de una pieza opcional (omitida si es peón), seguida obligatoriamente de la columna de destino y, finalmente, la fila de destino.

Ejemplos de cadenas válidas dentro de este subconjunto son $e4$, $Nf3$, $Qa5$ y $h8$.

Cadenas como $Pe4$ (los peones no se representan con P), $4e$ (orden incorrecto) o $Bc9$ (fila fuera de rango) deben ser rechazadas.

Expresión Regular (Regex) y Autómata Finito Determinístico (AFD)

Expresión regular

De acuerdo con la regla sintáctica descrita, la expresión regular que modela todas las cadenas válidas del subconjunto es:

$$Regex = (K|Q|R|B|N)?(a|b|c|d|e|f|g|h)(1|2|3|4|5|6|7|8)$$

Esta expresión se lee como: una pieza opcional (grupo con ? que actúa como cero o una ocurrencia), seguida exactamente de un carácter de columna y, a continuación, uno de fila. El operador de concatenación implícita y la unión (alternancia) reflejan exactamente la estructura secuencial del movimiento.

Autómata Finito Determinístico

A partir de la expresión regular se construye un AFD que realiza el reconocimiento.

El autómata se define formalmente mediante la tupla $M = (Q, \Sigma, \delta, q_0, F)$:

Conjunto de estados (Q): $\{q_0, q_1, q_2, q_3, q_{error}\}$.

Alfabeto (Σ): $\Sigma_p \cup \Sigma_F \cup \Sigma_R$.

Estado inicial (q_0): q_0 .

Estados de aceptación (F): q_3 .

El estado q_{error} actúa como sumidero: cualquier transición no definida explícitamente conduce a él, y una vez en q_{error} el autómata permanece allí para el resto de la cadena, rechazándola al final.

La función de transición δ se resume en la tabla siguiente:

Estado actual Entrada: Pieza (Σ_P) Entrada: Columna (Σ_F) Entrada: Fila (Σ_R)

$\rightarrow q_0$	q_1	q_2	q_{error}
q_1	q_{error}	q_2	q_{error}
q_2	q_{error}	q_{error}	q_3
q_3	q_{error}	q_{error}	q_{error}
q_{error}	q_{error}	q_{error}	q_{error}

El comportamiento del AFD se explica por sus estados:

- q_0 : estado inicial, no se ha leído ningún carácter. Si llega una pieza se avanza a q_1 ; si llega una columna (movimiento de peón) se salta a q_2 ; cualquier otra entrada es ilegal.
- q_1 : ya se leyó una pieza. La única entrada legal es una columna, que conduce a q_2 .
- q_2 : se ha reconocido la pieza (si la hubo) y la columna. La única entrada legal es una fila, que lleva al estado final q_3 .
- q_3 : estado de aceptación. Cualquier carácter adicional hace que el autómata caiga en q_{error} , ya que la cadena debe terminar exactamente después de la fila.
- q_{error} : estado sumidero que consume cualquier carácter y rechaza al final.

Este diseño garantiza que se acepten exclusivamente las cadenas que se ajustan a la expresión regular, procesando la entrada en una sola pasada y sin ambigüedad.

Implementación en Python y validación

Para demostrar el funcionamiento práctico del AFD, se desarrolló un script interactivo en Python que simula las transiciones estado por estado. El programa solicita movimientos al usuario en tiempo real y responde si son aceptados o rechazados.

```
def validador_pgn_basico(movimiento):

    """

    Simula un Autómata Finito Determinístico (AFD) para validar

    un subconjunto básico de la notación PGN de ajedrez.

    """

    piezas = {'K', 'Q', 'R', 'B', 'N'}

    columnas = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}

    filas = {'1', '2', '3', '4', '5', '6', '7', '8'}

    estado_actual = 'q0'

    for caracter in movimiento:

        if estado_actual == 'q0':

            if caracter in piezas:

                estado_actual = 'q1'

            elif caracter in columnas:

                estado_actual = 'q2'

            else:
```

```

        estado_actual = 'q_error'
    elif estado_actual == 'q1':
        if caracter in columnas:
            estado_actual = 'q2'
        else:
            estado_actual = 'q_error'
    elif estado_actual == 'q2':
        if caracter in filas:
            estado_actual = 'q3'
        else:
            estado_actual = 'q_error'
    elif estado_actual == 'q3':
        estado_actual = 'q_error'
    if estado_actual == 'q_error':
        break

    return estado_actual == 'q3'

if __name__ == "__main__":
    print("=====")
    print("    VALIDADOR LÉXICO PGN (SUBCONJUNTO BÁSICO)    ")
    print("=====")
    print("Escriba el movimiento a evaluar.")
    print("Escriba 'salir' para terminar la ejecución.\n")
    while True:
        mov = input("Ingrese un movimiento: ").strip()

```

```

if mov.lower() == 'salir':

    print("Cerrando el validador...")

    break

if not mov:

    continue

if validador_pgn_basico(mov):

    print(f" -> [ÉXITO] El movimiento '{mov}' es VÁLIDO y fue aceptado.\n")

else:

    print(f" -> [ERROR] El movimiento '{mov}' es INVÁLIDO y fue rechazado.\n")

```

La función `validador_pgn_basico` sigue fielmente la tabla de transiciones. Por cada carácter consulta el estado actual y decide el siguiente estado, implementando así el mismo comportamiento secuencial del AFD. Si en algún momento se llega a ' q_{error} ', el bucle se interrumpe y la cadena se rechaza. Solo se acepta si al final del recorrido el estado es ' q_3 '.

Conclusión

El desarrollo del presente trabajo ha permitido consolidar los conceptos fundamentales de la teoría de lenguajes formales y su aplicación directa en la construcción de compiladores. Se ha demostrado que la comprensión de la jerarquía de gramáticas es crucial para seleccionar el enfoque de análisis adecuado para cada tipo de lenguaje. La implementación de casos prácticos, como los ejemplos en Java para cada tipo de gramática de Chomsky, ha facilitado la transición de conceptos abstractos a soluciones computacionales concretas, resaltando la relación directa entre la teoría y su realización en software.

Se ha puesto de manifiesto la importancia de la higiene de gramáticas a través del análisis de patologías como la ambigüedad, la recursividad por la izquierda y la factorización izquierda. La resolución de estos problemas mediante técnicas específicas, como la transformación de reglas y la factorización de prefijos comunes, resulta imperativa para garantizar la construcción de analizadores sintácticos deterministas y eficientes, como se ha demostrado con los scripts en JavaScript.

. La implementación de un AFD a partir de una expresión regular para validar movimientos simplificados ha validado la teoría de autómatas en un contexto real y verificable. El diseño del AFD en Python demuestra un proceso sistemático que garantiza un reconocimiento eficiente, secuencial y sin ambigüedad. En conclusión, este trabajo ha proporcionado una visión integral y práctica de cómo los fundamentos matemáticos de los lenguajes formales se traducen en herramientas fundamentales para la ingeniería de software, sirviendo como base para el desarrollo de compiladores y procesadores de lenguajes de dominio específico.

Referencias

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson Education.
- Sipser, M. (2013). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.
- Prusinkiewicz, P., & Lindenmayer, A. (1990). The Algorithmic Beauty of Plants. Springer-Verlag.
- Chomsky, N. (1956). Three models for the description of language. IRE Transactions on Information Theory, 2(3), 113–124.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.
- Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.